

Key words – Trace graph, Network Simulator ns-2, trace files analysis

Jarosław MAŁEK*, Kamil NOWAK

TRACE GRAPH – DATA PRESENTATION SYSTEM FOR NETWORK SIMULATOR NS-2

Abstract – This paper describes Network Simulator ns-2 trace files analysis problem. The simulator works under UNIX and Windows system platforms and is mainly used for network research. It is widely used all over the world, especially at universities. The main problem is that ns-2 doesn't provide any visualization options for simulation results (trace files) analysis. Because of this reason Trace graph – ns-2 data presentation system was created in year 2001 and has been developed for a year and a half. It is implemented in MATLAB and can be compiled to run standalone without MATLAB application installed. The system, among others, has capabilities to calculate many parameters characterizing network simulation, e.g. delay, RTT, processing time, to plot 250 various graphs, to save calculations results to text files and to process its own script files to do all the calculations automatically. Trace graph has become very popular and is used by many ns-2 users. A list of documents, which authors used the program for simulations results analysis, is included.

1. INTRODUCTION

In the past ten years computer networks have been significantly developed, especially the largest one – the Internet. Nowadays networks have more and more workstations, the transmission speed increases and the connections capabilities rise. Continuous networks development, higher and higher functionality requirements have created the need for tools that could monitor network transmissions and analyse them. There have been developed network simulators, which help to design and test various kinds of networks, protocols, etc. The main advantage of simulations is low cost of development and implementation in comparison to experimental tests in real environment.

This paper describes Trace graph – a data presentation system for Network Simulator ns-2. The simulator doesn't have any options implemented to analyse simulations results so it's hard to use it. Trace graph system provides many options for analysis, including 250 graphs and statistical reports. It is implemented in MATLAB 6.0 and can be compiled to run without MATLAB. Compiled versions for Linux and Windows systems are available for download at <http://www.geocities.com/tracegraph/>.

* Wrocław University of Technology, Institute of Control and Systems Engineering, Poland, jaroslaw_malek@interia.pl

2. NETWORK SIMULATOR NS-2

The Network Simulator ns-2 is a discrete event simulator, which means it simulates such events as sending, receiving, forwarding and dropping packets. The events are sorted by time (seconds) in ascending order. The simulator works under UNIX and Windows system platforms and is available at <http://www.isi.edu/nsnam/ns/> for free. It is implemented in C++. For simulation scenario and network topology creation it uses OTcl (*Object Tool Command Language*). To create new objects, protocols, and routing algorithms or to modify them ns-2 C++ source code has to be changed. The simulator supports wired, wireless and satellite networks. Figure 1 shows the ns-2 system architecture on a simple block diagram. One of the blocks represents Trace graph. With ns-2 comes Network Animator – a visualization tool for packets flows. It only shows generated packets movement as an animation, so this is not enough to do simulation results analysis. The simulator users have to create their own programs to process the results – trace files [1]. The ns-2 output data contains a lot of complicated information about simulation. There are a few trace file formats and each of them can have various versions, so it can be hard to find out how to extract necessary data. This is the reason why Trace graph is so useful. It makes trace file processing very simple.

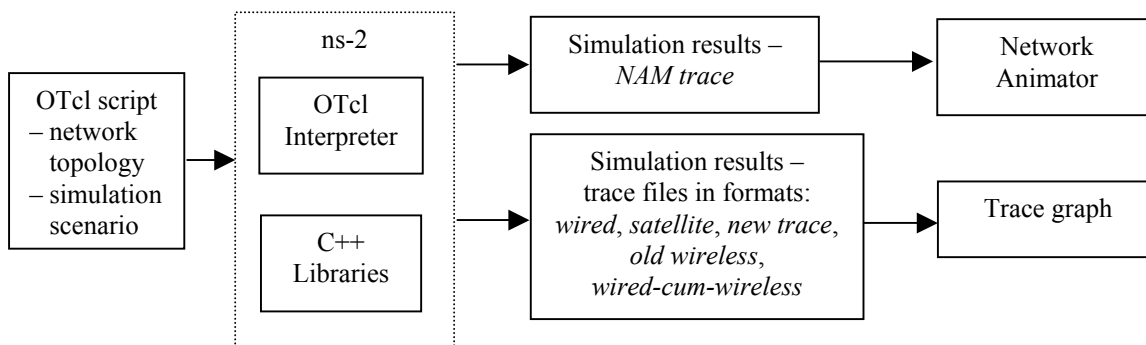


Fig. 1. Simulation stages

3. QUALITY DETERMINANTS FOR SIMULATION RESULTS ANALYSIS

For simulation results analysis the following quality determinants are used in Trace graph system:

- delay = packet receive time at destination node – packet send time at source node,
- Round Trip Time (RTT) = acknowledge (ACK) packet receive time at source node – packet send time at source node,
- processing time = packet forward time at intermediate node – packet receive time at intermediate node,
- jitter = absolute value of (delay of packet i+1 – delay of packet i),

where $i = 1..number\ of\ sent\ packets - 1$, [2],

- throughput = number of generated/sent/received/forwarded/dropped packets or bits in a certain time interval, e.g. 1000 bits/1 second.

4. SIMULATION RESULTS PROCESSING

4.1. TRACE FILES PROCESSING

Trace graph supports the following ns-2 trace file formats:

- wired,
- satellite,
- wireless (old and new trace),
- wired-cum-wireless.

Trace file loading stage is divided into 4 stages:

- automatic trace file format recognition using the first n file lines, where n is specified in Trace graph configuration file, if the format cannot be recognized it can be specified manually,
- trace file parsing to extract necessary simulation data which is saved to a temporary file, trace files can contain much more data than is needed by the system, so unnecessary information is omitted to speed up trace file loading,
- temporary file loading,
- constants calculations (packets types, packets sizes, flows IDs, trace levels, number of nodes, simulation time) – in order to speed up data processing.

Wireless and wired-cum-wireless trace files are parsed and saved in Trace graph format (see table below).

Trace graph format

Type	Value
	%c %d %i %s %i %s %i %i %i %i %i %i
char	event (s – send, r – receive, f – forward, d – drop)
double	event time in seconds
int	current node number
string	trace level (AGT – agent, RTR – router, MAC)
int	unique packet ID
string	packet type + 15 – (packet type length) spaces
int	packet size
int	previous node number
int	source node number
int	destination node number
int	next node number
int	packet sequence number (equals -1 if not present)

Every trace file not only ns-2 trace file can be converted to Trace graph format and loaded by the system. A big trace file loading can take a long time, e.g. 20 minutes. That's why

there is an option to save any loaded trace file in MATLAB's binary format (*.mat* extension). This enables to load them again in a few seconds.

4.2. DATA PRESENTATION

Simulation results can be analysed with the following graphs:

- cumulative sums of numbers of generated/sent/received/forwarded/dropped packets or bytes vs. time,
- throughput of generating/sending/receiving/forwarding/dropping packets or bits vs. time,
- packet size vs. minimal/average/maximal throughput of generating/sending/receiving/forwarding/dropping packets or bits,
- packet ID or send/receive event time vs. jitter,
- throughput of sending/receiving bits vs. minimal/average/maximal delay,
- throughput of sending/receiving bits vs. minimal/average/maximal jitter,
- throughput of forwarding/receiving bits vs. minimal/average/maximal processing time,
- throughput of sending bits vs. minimal/average/maximal Round Trip Time,
- send/receive event time vs. number of intermediate nodes forwarding/receiving packet,
- packet ID vs. generate/send/receive/forward/drop event time,
- packet sequence number vs. generate/send/receive/forward/drop event time,
- packet size vs. minimal/average/maximal delay,
- packet size vs. minimal/average/maximal processing time,
- packet size vs. minimal/average/maximal Round Trip time,
- delay/jitter/processing time/RTT cumulative distribution,
- delay/jitter/processing time/RTT frequency distribution,
- packet ID or send/receive event time vs. delay,
- packet ID or forward/receive event time vs. processing time,
- packet ID or send/receive event time vs. RTT,
- 3D graphs – numbers of generated/sent/forwarded/received/dropped/lost packets or bytes at all the nodes.

Besides Graphs window (Fig. 2.) there is Network Information window (Fig. 3.) that enables to calculate and show basic parameters describing simulation, e.g. number of generated packets, delays, RTT, processing times. All this information can also be obtained using only Graphs window. All the calculations done by the system can concern the whole simulated network or two specific nodes. All the results can be filtered by packets types, packets sizes, flows IDs, trace levels, time interval and nodes numbers – these options are available in the main program window. All the plotted data can be saved to a text file (as X, Y, Z columns separated with spaces respectively for each axis values). Moreover statistics like minimal/average/maximal value, median and standard deviation for each axis values can be saved to other text files. Saved graphs can be loaded and plotted without repeating calculations. Every text file that contains 2 or 3 columns with numbers can be treated as

a graph data. This enables to use Trace graph for any graph plotting. Graphs can also be saved in tiff or jpeg format.

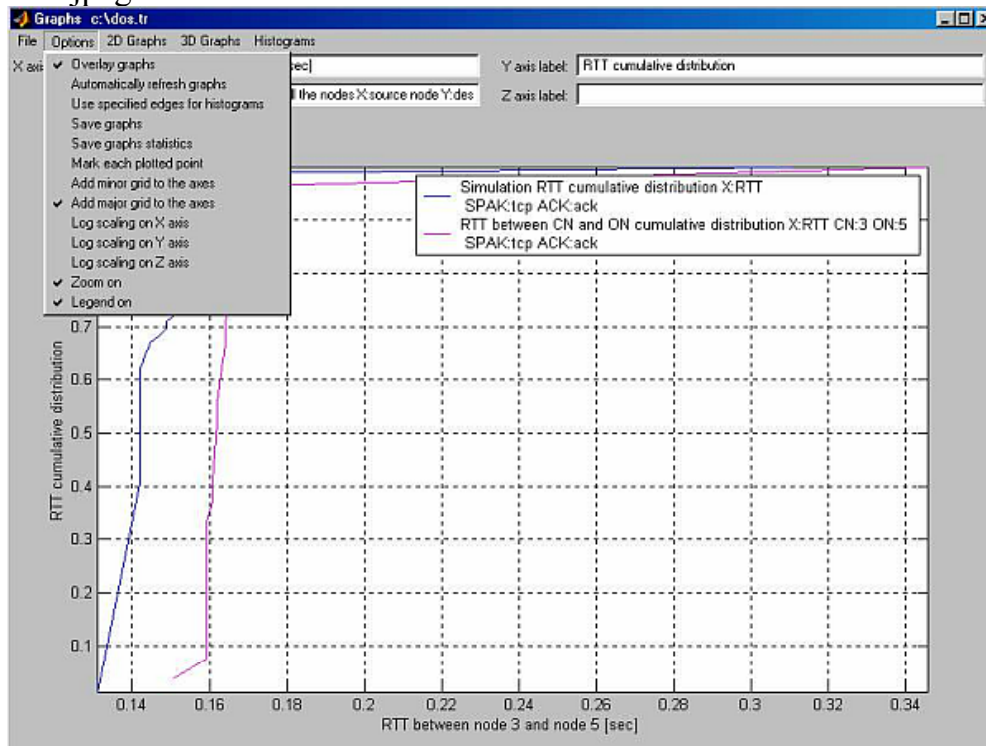


Fig. 2. Graphs window

Simulation information:		Simulation End2End delays in seconds:	
Simulation length in seconds:	1.574507	Minimal delay (CN,ON,PID):	0.065464 (9,10,20)
Number of nodes:	11	Maximal delay (CN,ON,PID):	0.102147 (3,5,316)
Number of sending nodes:	4	Average delay:	0.0836610625
Number of receiving nodes:	4	Delays between current and other node in seconds:	
Number of generated packets:	297	Minimal delay (PID):	0.075357 (4)
Number of sent packets:	297	Maximal delay (PID):	0.102147 (316)
Number of forwarded packets:	297	Average delay:	0.09038684091
Number of dropped packets:	9	Average numbers of intermediate nodes for the whole network:	
Number of lost packets:	9	Average number of nodes receiving packets:	3
Minimal packet size:	40	Average number of nodes forwarding packets:	3
Maximal packet size:	1000	Average numbers of intermediate nodes between current and other node:	
Average packet size:	986.9388	Average number of nodes receiving packets:	3
Number of sent bytes:	293160	Average number of nodes forwarding packets:	3
Number of forwarded bytes:	861480	Simulation processing times at intermediate nodes in seconds:	
Number of dropped bytes:	9000	Minimal (node,PID):	0 (1,0)
Packets dropping nodes:	0	Maximal (node,PID):	0.011174 (1,322)
Current node information:		Average:	0.001420712486
Number of generated packets:	47	Processing times at current node in seconds:	
Number of sent packets:	47	Minimal (PID):	N/A
Number of forwarded packets:	0	Maximal (PID):	N/A
Number of received packets:	0	Average:	N/A
Number of dropped packets:	0	Simulation Round Trip Times in seconds:	
Number of lost packets:	3	Minimal RTT (CN,ON,SPID):	0.130928 (8,7,0)
Number of sent bytes:	46040	Maximal RTT (CN,ON,SPID):	0.310928 (8,7,322)
Number of forwarded bytes:	0	Average RTT:	0.1474974937
Number of received bytes:	0	Round Trip Times between current and other node in seconds:	
Number of dropped bytes:	0	Minimal RTT (SPID):	0.150715 (4)
Minimal packet size:	40	Maximal RTT (SPID):	0.345462 (191)
Maximal packet size:	1000	Average RTT:	0.1693990741
Average packet size:	979.5745		

Fig. 3. Network Information window

4.3 SCRIPT FILES

To process a trace or a mat file automatically a Trace graph script file has to be written. This option is useful if there is a lot of trace files to analyse or trace files are big and it takes a lot of time to process them. Trace graph makes commands specified in a script file automatically. All the analysis results are saved to text files.

Script file can be passed as a command line parameter, for example:

```
trgraph c:\trgraph\out.tr -script c:\samplescript.txt
```

An example script file:

SAVE MAT FILE: ON

SAVE GRAPHS TO JPEG: ON

SAVE GRAPHS STATISTICS: ON

GRAPHS LEGEND: OFF

RESET OPTIONS: ON

CLOSE PROGRAM WHEN FINISHED: ON

GRAPHS:

IDs of all the generated packets,PT: cbr,ST: 1.1,ET: 3.3,MARKPOINTS,MINORGRID

%Cumulative sum of numbers of generated packets at current node,PT: tcp,CN: 2

CN2ON delays frequency distribution,HISTEDGES: 0.01 0.02 0.03 0.04,PT: cbr,DC

NETWORK INFORMATION:

Simulation information,PT: cbr

Current node information,FID: 1

Options explanation:

- *SAVE MAT FILE* – saves a trace file when loaded to mat file,
- *SAVE GRAPHS TO JPEG* – saves all the plotted graphs to jpeg files not only to text files,
- *SAVE GRAPHS STATISTICS* – saves all the plotted graphs statistics to text files,
- *GRAPHS LEGEND* – enables/disables legend on the graphs,
- *RESET OPTIONS* – turns off all the system options (like Packet type, Flow ID, Other node, Time interval, etc.) after each processed line from the script file,
- *CLOSE PROGRAM WHEN FINISHED* - closes (or not) Trace graph when analysis is done.

Lines starting with % are not processed. Graphs and network information names, e.g. *IDs of all the generated packets* have to be the same as they appear in the system menus. This enables to add new graphs to Trace graph and plot them during script file processing without adding additional source code to system function which process scripts.

5. PRACTICAL APPLICATIONS OF THE SYSTEM

Trace graph has been used by at least few thousands ns-2 users at universities and companies all over the world for over a year and half. Many documents (thesis works, conference articles, etc.) referring to Trace graph have been created. Below there is a list of selected documents which authors used Trace graph to analyse simulations results.

- R. Lauder, *Quality of Service Provisioning over Broadband Satellite Networks*, Trinity College Dublin, Ireland, 2003
- V. Tobi, *Internet traffic engineering using MPLS*, University of Bucharest, Romania, 2003
- K. Ramachandra, *Evaluating the Performance of Various Architectures for Wireless Ad Hoc Networks*, University of Nebraska, USA, 2003
- S. Gupta, *Simulating IEEE 802.11 with Dynamic Source Routing (DSR) in NS*, Indian Institute of Information Technology, India, 2003
- P. Wagstrom, *Multicast Simulation Using NS-2*, Illinois Institute of Technology, USA, 2003
- A. Sathiaselan, T. Radzik, *RD-TCP: Reorder Detecting TCP*, King's College London, UK, 2003
- H. J. Steinhauer, S. Ohlsson, *Comparison of Combined Dispatching Strategy for Two Different Queues to a Single Queue System*, Linköping University, Sweden, 2002
- J. R. N. Gonzalez, *Estudio y Aplicacion Practica del Protocolo RSVP Mediante el Network Simulator 2*, Escuela Universitaria de Ingenieria Tecnica de Telecomunicacion, Spain, 2003
- S. Du, *TCP Fairness*, University of New South Wales, Australia, 2002
- A. Vidali, *Realizzazione di un simulatore per lo studio dell'interferezna tra reti Wi-Fi e Bluetooth*, Universita degli Studi di Trento, Italy, 2003
- G. V. Kinal, *PRELIMINARY APPLICATION OF NS NETWORK SIMULATOR FOR IP-BASED DEFENSE SATELLITE SYSTEMS*, Femmecomp, Inc., USA, 2003
- N. F. N. Chebib, *Multicast routing protocols*, St. Joseph University, Lebanon, 2003
- S. R. Tuladhar, *Routing protocols in ad hoc networks*, Kathmandu University, Nepal, 2003
- N. M. Ha, *A comparison for improving consistency in scalable network using simulation*, Nation University, Vietnam, 2003
- M. Stojanovic, *An Approach for Planning and Design of QoS Enabled IP Networks*, Mihailo Pupin Institute, Serbia, TELSIS IEEE conference, 2003
- M. Portnoi, R. G. Bezerra de Araujo, *Network Simulator - Visao Geral da Ferramenta de Simulacao de Redes*, Universidade Salvador, Brasil, 2003
- M. Portnoi, R. G. Bezerra de Araujo, *Ferramenta de Simulacao de Redes de Computadores*, Universidade Salvador, Brasil, 2003
- C. Budakoglu, *Research Proceeding Report*, University of Victoria, Canada, 2003
- S. Parameswaran, *Optimal Selection of Nodes in a Multihop Wireless Ad Hoc Network*, Mississippi State University, USA, CIIT conference, 2003
- S. Al-Sharaeh, *Ad Hoc Routing Techniques Performance analysis*, King Fahd University of Petroleum and Minerals, Saudi Arabia, 2003

- S. L. Ramírez, *Redes ad-hoc para comunicaciones móviles*, University of Málaga, Spain, 2003
- L. R. L. Vega, L. L. Velarde, F. A. O. Garcia, L. G. Medina, P. Manzoni, *Simulacion y Evaluacion de las Prestaciones de una Red Ad Hoc Inalambrica en un Campus Universitario con Trafico de Datos en Tiempo Real y No Real con el Modelo Swan*, Universitat Politecnica de Valencia, Spain, CIICC congress, 2003
- L. R. L. Vega, L. L. Velarde, F. A. O. Garcia, L. G. Medina, P. Manzoni, *Analisis de la calidad de servicio (QoS) de una Red Ad Hoc Inalambrica con servicios diferenciados en el simulador NS2*, Universitat Politecnica de Valencia, Spain, CIICC congress, 2003
- P. Clark, *Modelling of Dynamic Routing over an Ad-Hoc Network*, Napier University, UK, 2003
- R. A. Pasko, *Comparison of the Protocol Independent Multicast Routing Protocol's Dense and Sparse Modes using the NS2 Network Simulator*, Illinois Institute of Technology, USA, 2003
- U. Birnbacher, *Simulation of multiple access algorithms for satellite based communication networks*, University of Technology Graz, Austria, 2003
- A. Georges, *TCP over Wireless Networks*, FSA-UCL, France, 2003
- D. Salman, *Remote-Experimentation via Internet: Simulation Role and QoS-Adaptation (Ph.D)*, University of Evry, France, 2003
- J. Cannau, *Evaluation de performance d'une amélioration de TCP Westwood*, Université Libre de Bruxelles, Belgium, 2003
- A. Koliver, J. M. Farines, C. E. Santin, *Discussion and Preliminary of a Decentralized Scheme for Congestion Control*, Universidade de Caxias do Sul, Brasil, 2003
- I. Safitri, *Comparison of DSDV and AODV Routing Protocol for 4G Mobile Communication*, Bandung Institute of Technology, Indonesia, 2003

6. EXPANDING SYSTEM CAPABILITIES

Trace graph architecture enables implementing new system functions very easy. For example adding a new graph can take only 10 minutes. The system could be expanded to read other trace file formats like real network traces, e.g. a trace format converter could be created for conversion to Trace graph format. Nodes movements with packets flows 2D/3D visualization could be added. More parameters from trace files could be used for new graphs implementation. MATLAB environment enables to develop the system very easy and very fast.

REFERENCES

- [1] <http://www.eecs.wsu.edu/~rgriswol/NS2/ns2-trace-formats.html>, Richard Griswold, *NS-2 Trace Formats*.
- [2] <http://nms.lcs.mit.edu/~hari/papers/CS294/paper/node5.html>, Elan Amir, *Unreliable Transport*.